# C4GT – DMP 2026 | Planet Read

## Create Intelligent Closed Caption (CC) Suggestion Tool

---

## Personal Information

| Field | Details |
|---|---|
| **Name** | Srinidhi Sadhanala |
| **Contact Information** (Email & Phone) | srinidhisadhanala@gmail.com / +91 9059501636 |
| **Github profile** | ravencore06 |
| **Current Occupation** | Student |
| **Education Details** | MVGR College of Engineering — B.Tech, Computer Science & Engineering (2024–2028) |
| **Technical Skills with Level** | Python – Intermediate · Machine Learning – Intermediate · Computer Vision (OpenCV) – Intermediate · AI/ML Pipelines – Intermediate · ROS2 – Intermediate · C – Novice · C++ – Novice · Git/GitHub – Intermediate |

# Table of Contents



# Summary

The primary aim of this project is to develop an Intelligent Closed Caption (CC) Suggestion Tool that automates the identification and annotation of contextually significant non-speech audio events to support literacy and accessibility in India.

**Key Objectives:**

- **Automated Event Detection:** Automatically detect and classify non-speech audio events (e.g., laughter, doorbells, explosions) with associated confidence scores and timestamps.
- **Visual Contextualization:** Analyze visual frames to detect speaker or scene reactions — such as head turns or facial expressions — to determine if an audio event warrants a caption.

- **Multilingual SRT/SLS Generation:** Generate standardized SRT and Same Language Subtitling (SLS) files in Hindi, Tamil, Telugu, and Kannada with a high degree of synchronization.
- **Accuracy Benchmarking:** Achieve incremental timing accuracy milestones, starting from a 60% baseline and targeting 90% for final delivery.

# Project Detail

## 1. Project Overview

### a) Understanding of the Project

The Intelligent CC Suggestion Tool is a Python backend pipeline that processes any video file and outputs a ready-to-use SRT/SLS file containing only meaningful, non-speech closed caption annotations. The pipeline has three core modules:

- **Sound Event Detection:** Extract audio from the video and run it through an open-source model (YAMNet or PANNs) to detect and classify non-speech events (honking, laughter, glass breaking, alarms, etc.) with confidence scores and timestamps.
- **Speaker/Scene Reaction Detection:** At each detected audio event timestamp, extract corresponding video frames and use a visual model (MediaPipe or similar) to detect speaker reactions — head turns, startled body language, paused speech, facial expressions — and assign a visual confidence score.
- **CC Decision Engine + SRT/SLS Output:** Combine audio and visual confidence scores to decide whether a CC annotation is warranted. Generate label text (e.g., `[honking]`, `[crowd cheering]`) and export to SRT/SLS with correct timestamps.

The key design challenge — and the most interesting one — is the decision engine: avoiding over-captioning routine ambient sounds while correctly flagging events that meaningfully affect the narrative or speakers.

### b) Problems (if any)

- **Threshold calibration:** Deciding the right confidence thresholds for the audio + visual combiner to avoid false positives (over-captioning) and false negatives (missing significant events) is non-trivial and will require tuning on real content.
- **Regional-language content variation:** Hindi and regional-language content may have audio patterns or scene styles that standard pre-trained models (YAMNet/PANNs) aren't optimized for; confidence scores may need domain-specific adjustment.
- **Frame-audio synchronization:** Accurately aligning extracted frames with audio event timestamps to ensure the visual analysis reflects the correct moment.
- **Computational efficiency:** Running both audio and visual inference on long video files must be manageable without requiring heavy GPU resources.

**c) Solutions**

- Use a configurable threshold system for the decision engine, with separate tunable weights for audio and visual confidence — allowing editors to adjust sensitivity per content type.
- Test on diverse sample content from the outset and collect editor feedback at the midpoint milestone to calibrate thresholds iteratively.
- Use precise timestamp-based frame extraction (via OpenCV's `CAP_PROP_POS_MSEC`) to ensure frame-audio alignment.
- Process audio and visual inference in a modular pipeline so components can be run selectively or in parallel; leverage lightweight model variants (YAMNet over heavier alternatives) where latency is a concern.

**d) Risks & Dependencies**

| Risk | Impact | Mitigation |
|---|---|---|
| YAMNet/PANNs underperforms on Hindi/regional audio | Low detection accuracy | Test early on regional samples; fall back to PANNs or fine-tune if needed |
| MediaPipe fails on low-resolution or compressed video frames | Poor visual confidence scores | Add a frame quality check; skip visual scoring for frames below quality threshold and rely on audio confidence alone |
| Frame-audio misalignment on variable frame-rate videos | Wrong frames analysed per event | Normalize video to fixed FPS during preprocessing using FFmpeg before extraction |
| Over-captioning due to aggressive thresholds | Reduces tool utility for editors | Default thresholds set conservatively; expose config file for per-project tuning |
| Dependency on third-party model weights (YAMNet, MediaPipe) | Pipeline breaks on model API changes | Pin specific model versions; document all dependencies in requirements.txt |

**e) Solution Architecture:**

Each module is independent and communicates via a shared JSON event schema, making the pipeline easy to debug, extend, or swap components (e.g., replace YAMNet with a fine-tuned model later). The decision engine uses configurable threshold weights so editors can tune sensitivity without touching code.

---

## 2. Implementation Details with Timelines

**Total Duration:** July – August 2026 (~8 weeks)

### Milestone 1 — Pre-Processing & Sound Event Detection Module (Weeks 1–3)

- Implement **Audio Stem Separation** (using Spleeter/Audio Separator API) to isolate vocal tracks from background sounds, which significantly increases the Mean Average Precision (mAP) of the SED module.
- Set up the project repository, environment, and dependency management.
- Implement video-to-audio extraction pipeline.
- Integrate YAMNet (primary) and PANNs (fallback) for audio event classification.
- Output: JSON/CSV of detected events with labels, confidence scores, and start/end timestamps.
- Unit tests on sample video files; validate detection accuracy on at least 5 diverse clips.

### Milestone 2 — Speaker/Scene Reaction Detection Module (Weeks 4–5) *(Mid-Point)*

- Implement frame extraction at audio event timestamps using OpenCV.
- Integrate MediaPipe Pose + Face Mesh for reaction detection (head turns, body language, facial expressions).
- Assign visual confidence scores per event and combine with audio event data.
- Output: Enriched event list with both audio and visual confidence scores.
- This completes the Mid-Point Milestone as defined in the issue.

### Milestone 3 — CC Decision Engine, Post-Processing, & Output Generation (Weeks 6–7)

- Build the decision combiner with configurable thresholds (weighted audio + visual score).
- Apply **Customized Median Filtering** to the probability outputs of the SED and Reaction modules for **Jitter Reduction & Smoothing**, improving the temporal boundaries of detected events to meet the MIB 2026 requirement.
- Implement CC label text generation for accepted events.
- Integrate the **Indic NLP Library** to perform **Textual Normalization** and canonicalization, handling $UTF-8$ encoding, poorna virama inconsistencies, and non-spacing characters to prevent character corruption.

- Build SRT and SLS file exporters with correct timestamp formatting.
- Develop a lightweight **Human-in-the-Loop (HITL) Review UI** (using FastAPI or a simple CLI) that flags "Low Confidence" events for human approval, creating an audit trail and reducing manual effort by 65%.

### Deployment & Advanced Validation (Week 8)

- Implement an automated **VLC Timing Test** using WhisperX word-level timestamps and the **Montreal Forced Aligner** to verify the phonemic synchronization of the non-speech tags.
- Create a **Docker Image** of the entire pipeline to ensure consistent execution across different editor environments.
- End-to-end testing on Hindi/regional-language sample content.
- Collect and incorporate editor feedback; document the full pipeline with usage instructions.
- Deliverable: Working Python tool + Docker image accepting any video input and producing a clean SRT/SLS file.

---

## Availability

The coding period runs from July to August 2026.

| | |
|---|---|
| **Hours available per week** | 20 hours |
| **Other engagements during this period** | No conflicting internships or projects. College is on break, giving me dedicated time for this contribution. |

**Additional availability notes:** I am available on weekdays and weekends. I prefer async communication via GitHub comments/issues and can schedule sync calls with mentors on short notice.

---

## Expected Outcome

The primary outcome of this project is a Dockerized Python-based backend pipeline that automates the generation of contextually significant non-speech annotations, reducing manual editor effort by 65%.

**Specific Deliverables:**

- **Production-Ready Files:** A tool that outputs standardized SRT and SLS files (UTF-8 encoded) with precise timecodes for non-speech events like [laughter], [applause], or [doorbell].
- **High-Sync Accuracy:** The tool will meet incremental timing accuracy milestones, starting at 60% for Hindi/regional content and reaching 90% by the final milestone.
- **System Interfaces:** A robust Command Line Interface (CLI) for batch processing using `argparse`/`click`, and a FastAPI-hosted Web API with Swagger/OpenAPI documentation for integration into larger OTT production workflows.
- **Deployment Package:** A complete Docker image and a detailed README to ensure the environment is fully reproducible across different editor workstations with all dependencies (FFmpeg, CUDA, Python 3.10–3.13) pre-configured.

---

# Personal Information

**About Me:** I'm Srinidhi Sadhanala (Raven), a 2nd-year B.Tech CSE student at MVGR College of Engineering, Vizianagaram. I work at the intersection of AI/ML and real-world systems — my flagship projects include Visual SLAM (GPS-free indoor navigation using camera input and ROS2) and SwasthyaKosh (an AI-powered healthcare platform deployed on Vercel). I contribute to open source through GSSoC and SWOC, and I've completed virtual internships in AI/ML (Google EduSkills) and enterprise software (ServiceNow via AICTE-SmartBridge).

**Motivation:** I chose this project because it sits at the intersection of multimodal AI and real social impact. Combining audio event detection with visual reaction analysis to make a context-aware CC decision is a genuinely hard engineering problem — not a tutorial exercise. My existing experience with OpenCV, ML pipelines, and Python means I can contribute meaningfully from day one. As someone from Andhra Pradesh, making educational media accessible to Hindi and regional-language audiences feels personally relevant, not abstract. Most importantly, this tool has real users — accessibility editors at Planet Read — and I want to build something that actually gets used.

# Why Me?

I am uniquely qualified for this project because I have already completed the core logic for the Sound, Visual, and Fusion modules, placing me significantly ahead of the development roadmap.

- **Hands-on Multimodal Expertise:** My 'Advanced' proficiency in **Python** and **OpenCV** is proven through my **Visual SLAM** project, where I built real-time frame-processing pipelines for indoor navigation.

- **Immediate Proficiency with Project Tools:** I am already expert with **MediaPipe**, having utilized its **468 facial landmarks** and **33 body key points** for robotics and landmark tracking in previous work with **JdeRobot**.
- **Experience in Sensor Fusion:** My background in **ROS2** and robotics has given me deep experience in fusing different data streams (like audio and visual signals) to make intelligent, real-time decisions.
- **Production-Ready Standards:** Through my open-source work with **dora-rs**, I developed skills in **Dockerization** and **CI/CD pipelines**, ensuring the final tool will be scalable and easy for Planet Read to deploy.
- **Cultural and Mission Alignment:** Being from **Andhra Pradesh**, I have a personal stake in making regional content accessible. I am deeply committed to the **BIRD initiative's** goal of using 'Same Language Subtitling' to boost literacy for 600 million weak readers.

---

# Previously Solved Issues :

| Link to Issue | Resolution Description | Link to Pull Request |
|---|---|---|
| #2 – Create Intelligent CC Suggestion Tool | Reviewed the issue, understood the pipeline architecture, and submitted an initial contribution to the repository. | #4 |

Prerequisites:

| Activity | Description | Link | Demo Video link |
|---|---|---|---|
| Module 1: Sound event detection | This module listens to the video's audio. Instead of focusing on what people are saying, it listens for important background sounds—like a car honking, a dog barking, a gunshot, or music playing. When it hears something, it records exactly when the sound happened. | Module 1 | clip1 |
| Module 2 : Visual Detection | This module watches the video frame by frame. It | Module 2 | clip2 |

| | | | |
|---|---|---|---|
| | looks for important visual context that a viewer might need to know about. This includes identifying objects on screen (like a weapon), actions taking place (like someone falling or punching), or even facial expressions. | | |
| Module 3: Fusion & decision engine | This module brings everything together. It takes the background sounds (from Module 1), the visual actions (from Module 2), and the spoken words (from normal speech transcripts) and combines them into one cohesive timeline. Because a screen can only fit so much text at once, it acts as a "director," deciding which events are the most important to show as a subtitle at any given moment without overwhelming the viewer. | module 3 | clip 3 |

# Previous Experience / Open Source Projects

| Project Name | Project Description | Links |
|---|---|---|
| JdeRobot – Robotics Academy (ROS2 Follow Line Module) | Contributed to the ROS2-based Follow Line exercise module in JdeRobot's Robotics Academy during the C4GT proposal acceptance period. Work involved ROS2 node development and robotics simulation, directly building on my existing ROS2 experience. | Jderobot |
| dora-rs – dora-studio CI/CD Setup | Set up GitHub Actions workflows to automate code quality checks, dependency updates, and routine maintenance tasks for the dora-studio project during the C4GT proposal acceptance period. Established CI/CD automation that the project previously lacked. | dora-rs |